```
{
    "id": 1,
    "name": "CBGB",
    "description": "Take it CJ! 1-2-3-4",
    "slug": "cbgb-1",
    "url": "/api/v1/bands/venue/1/",
}
```

Including the `url` field here may seem a little strange: you had to hit that URL to get this result, but the same serializer gets used in the listing code, which is where that value becomes useful. Are you ahead of me? Are you mumbling, *well, then let's add the listing code*? Who am I to argue? To add a bit of flair, the `Venue` listing endpoint also supports a filter. It takes an optional query parameter *name*, which, if given, filters the venues that begin with that value. The following listing shows the first attempt at this endpoint.

Listing 11.8 `Venue` object listing API end-point

```
# RiffMates/bands/api.py
...
@router.get("/venues/", response=list[VenueOut])
def venues(request, name=None):
    venues = Venue.objects.all()
    if name is not None:
        venues = venues.filter(name__istartswith=name)

    return venues
```

Support a query parameter called name.

If a name got provided, include only venues that start with it.

Endpoint view arguments get used by Ninja to support either URL arguments or query parameters. If a view argument has a corresponding brace-bracket surrounded value in the URL, it is a URL argument. Otherwise, it is treated as a query parameter. To make the query parameter optional, simply provide a default value to the function argument.

To filter listed results based on a name query parameter, you use the `QuerySet` object's `.filter()` method. Like with the `promoters()` endpoint, this endpoint function returns a list, indicated to Ninja through a Python type hint in the decorator. Make sure your development server is running, and visit http://localhost:8000/api/v1/bands/venues/ to see the results of your work.

11.2.3 Filtering your listings

A few paragraphs back, I wrote *first attempt*, did you catch it? Filtering lists is such a common activity that Ninja provides tools for it. For our `Venue` example, the tool is overkill: it takes more code with the tool than without. But if you want to support filters on multiple fields of an object, it can save you some work. To define a filter you build a `FilterSchema`, it is very similar to the other kinds of schemas you've seen so far, using Python type hints to declare the fields that can be filtered.